

Qwen3-Coder-14B Coding Agent Platform

Architecture, Deployment, Security & Cost Report

STATUS: FINAL | Prepared: August 2026

1. Executive Summary

This report documents the end-to-end design, deployment, and operating cost of a self-hosted coding agent platform built around **Qwen3-Coder-14B**, served on a single NVIDIA T4 GPU via **vLLM**. The platform includes a Claude Code-style CLI agent and a Perplexity-style web application, both authenticated through **Amazon Cognito** with federated Google and GitHub sign-in, and both routed through an AWS Lambda backend with human-in-the-loop safety controls for risky agent actions. The agent can also manage GitHub and GitLab repositories directly (branches, commits, pull/merge requests, issues) via REST API tools, with repository creation/deletion intentionally excluded. Deployment is fully automated via GitHub Actions and GitLab CI pipelines using OIDC federation, avoiding static AWS credentials entirely.

*This version of the report and its accompanying code package are marked **FINAL**.*

2. Model Selection

2.1 Best open-source coding models (2026)

Model	Coding Benchmark	Params (Active)	Min VRAM (Quantized)	License
Kimi K3	88.3 Terminal-Bench	2.8T (104B active)	Multi-GPU datacenter	Open weights
Qwen3.6-35B-A3B	73.4% SWE-bench Verified	35B (3B active)	~24GB (Q4)	Apache 2.0
Qwen3-Coder 32B	92.7% HumanEval	32B dense	~20.5GB (Q4_K_M)	Apache 2.0
Qwen2.5-Coder-32B	Beats larger models on code32B dense		~20GB (Q4)	Apache 2.0
Kimi-Dev-72B	60.4% SWE-bench	72B	Large (multi-GPU)	Open
Qwen3-Coder-Next	58.7% SWE-bench	80B MoE	48-49GB (Q4_K_M)	Open

2.2 Why Qwen3-Coder 14B for a single T4

A T4 GPU has 16GB of VRAM, which rules out any model at 32B parameters or larger even when quantized. **Qwen3-Coder 14B** at 4-bit AWQ quantization requires approximately 9.5GB of VRAM, leaving comfortable headroom for KV cache and moderate context lengths, and scores 85.2% on HumanEval. This makes it the strongest coding model that reliably fits a single T4 without aggressive offloading tricks that would hurt latency.

3. Serving Infrastructure

3.1 vLLM configuration

```
vllm serve Qwen/Qwen3-Coder-14B-Instruct-AWQ \
--quantization awq \
--gpu-memory-utilization 0.85 \
--max-model-len 8192 \
--dtype float16 \
--enable-auto-tool-choice \
--tool-call-parser qwen3_coder
```

T4's Turing architecture (compute capability 7.5) natively supports AWQ, GPTQ, Marlin, INT8 W8A8, GGUF, and bitsandbytes quantization kernels. FP8 is **not** supported on T4. The `--tool-call-parser qwen3_coder` flag is mandatory for structured tool calls to populate correctly in the OpenAI-compatible API response.

3.2 Deployment architecture

Layer	Technology	Purpose
Model serving	vLLM on T4 EC2 (g4dn.xlarge)	Hosts Qwen3-Coder-14B-AWQ, OpenAI-compatible API
Web backend	AWS Lambda (response streaming)	Auth, rate limiting, tool orchestration, EC2 auto-start
CLI client	Python + OpenAI SDK	Local agent loop with human-in-the-loop confirmation
Auth	Amazon Cognito	Federated Google + GitHub sign-in, OAuth 2.0 PKCE
State	DynamoDB	Pending human-in-the-loop actions, TTL-based expiry
Repo integrations	GitHub + GitLab REST API	Branches, commits, PRs/MRs, issues (no create/delete)
CI/CD	GitHub Actions + GitLab CI	OIDC-federated, credential-free automated deployment

4. Applications Built

4.1 CLI coding agent (Claude Code-style)

A terminal-based agent loop that reads/writes files and executes shell commands via tool-calling, authenticated through a local-loopback OAuth PKCE flow with tokens stored in the OS keyring rather than plaintext files.

4.2 Web application (Perplexity-style)

A browser-based chat interface with streaming token output, web search tool-calling for citations, Cognito Hosted UI login buttons for Google/GitHub, a live progress bar during EC2 cold-start, and an approve/deny UI for risky agent actions.

4.3 Human-in-the-loop safety gate

Both applications route `write_file`, `run_shell`, and all eight GitHub/GitLab repo-management tools through an explicit human confirmation step before execution, directly addressing OWASP's LLM Top 10 2026 guidance on Excessive Agency. Read-only tools (`read_file`, `web_search`) remain auto-approved since they carry no mutation risk.

4.4 GitHub / GitLab repository management

The agent can create branches, commit/update files, open pull requests or merge requests, and create issues on both platforms — all through REST API endpoints (`git/refs`, `contents`, `pulls`, `issues` on GitHub; `repository/branches`, `repository/files`, `merge_requests`, `issues` on GitLab), never by shelling out to

git, gh, or glab. Repository creation and deletion are intentionally not implemented.

4.5 EC2 auto-start with bounded startup budget

The T4 instance is stopped after 1 hour of inactivity and auto-started on the next incoming request. The Lambda backend polls instance state and vLLM health, streaming live progress percentages to the frontend progress bar. A hard 120-second startup budget is enforced.

5. Security & Standards Applied

Standard	What it governs	Implementation
OWASP LLM Top 10 (2026)	Prompt injection, excessive agency, tool safety	Tool allow-lists, human-in-the-loop approval for all mutating actions
OWASP API Security Top 10	Rate limiting, object-level authorization	Per-user rate limits, owns_conversation() ownership checks
Twelve-Factor App (2026)	Config, statelessness, structured logs	All secrets via env vars/Secrets Manager
OpenTelemetry	Distributed tracing, metrics	Instrumented across Lambda-to-vLLM call path
OAuth 2.0 PKCE	Auth code exchange security	Used by both CLI (loopback) and web app (Hosted UI)
OIDC federation (CI/CD)	Credential-free deployment	GitHub Actions and GitLab CI assume AWS IAM roles via short-lived tokens

5.1 Excessive Agency mitigation detail

This platform keeps privileged, state-changing actions — `write_file`, `run_shell`, and every GitHub/GitLab repo-management tool — gated behind explicit user approval. Repository creation/deletion was excluded entirely from the tool surface rather than merely gated.

6. CI/CD Automation

Both GitHub Actions and GitLab CI pipelines trigger automated deployment only on pushes to main, following an identical two-stage pattern (test then deploy). Both use IAM OIDC identity providers trusted specifically for the main branch, with no static AWS credentials stored in either platform.

7. Cost Analysis

7.1 Baseline monthly cost (100,000 requests/month, T4 EC2, 1-hour idle auto-stop)

Component	Bursty (6 GPU-hrs/day)	Typical (10 GPU-hrs/day)	Sparse (16 GPU-hrs/day)
EC2 g4dn.xlarge (T4) compute	\$94.68	\$157.80	\$252.48
AWS Lambda	\$6.69	\$6.69	\$6.69
EBS storage (125GB gp3)	\$12.50	\$12.50	\$12.50
Cognito	\$0.00	\$0.00	\$0.00
DynamoDB	\$0.17	\$0.17	\$0.17
Data transfer out	\$0.03	\$0.03	\$0.03
Total monthly cost	\$114.07	\$177.19	\$271.87

7.2 S3 + smaller EBS optimization

Setup	Storage	Monthly cost
Before: 125GB EBS gp3	OS + runtime + full model weights	\$10.00
After: 30GB EBS + 95GB S3	OS/runtime on EBS; weights on S3	\$4.58
Savings		\$5.42/month (\$65/year)

7.3 GPU/accelerator instance comparison (at 300 GPU-hours/month)

Instance	Accelerator	\$/hr	Monthly cost	Notes
inf1.xlarge	Inferentia1	\$0.228	\$68.40	Very limited LLM support
g4dn.xlarge (current)	T4 (16GB)	\$0.526	\$157.80	Native vLLM support
inf2.xlarge	Inferentia2 (32GB HBM)	\$0.7582	\$227.46	Needs Neuron SDK
g6.xlarge	L4 (24GB)	\$0.8724	\$261.72	Native vLLM support
g5.xlarge	A10G (24GB)	\$1.006	\$301.80	Fastest, native vLLM support

Conclusion: the T4 (g4dn.xlarge) remains the most cost-effective viable option at this scale.

8. Repository Management Tools Reference

Tool	Platform	Endpoint used	Approval?
github_create_branch	GitHub	POST /repos/{owner}/{repo}/git/refs	Yes
github_push_file	GitHub	PUT /repos/{owner}/{repo}/contents/{path}	Yes
github_open_pull_request	GitHub	POST /repos/{owner}/{repo}/pulls	Yes
github_create_issue	GitHub	POST /repos/{owner}/{repo}/issues	Yes
gitlab_create_branch	GitLab	POST /projects/:id/repository/branches	Yes
gitlab_push_file	GitLab	POST/PUT /projects/:id/repository/files/:path	Yes
gitlab_open_merge_request	GitLab	POST /projects/:id/merge_requests	Yes
gitlab_create_issue	GitLab	POST /projects/:id/issues	Yes

All eight tools call REST API endpoints exclusively and are gated behind human-in-the-loop approval. Repository creation and deletion were deliberately excluded.

9. Known Gaps and Recommendations

- GitHub OIDC compliance with Cognito needs manual verification.
- In-memory rate limiting must migrate to DynamoDB for multi-instance correctness.
- `run_shell` in the Lambda/web app path is a placeholder pending a controlled execution API.
- GitHub/GitLab tokens should be narrowed to fine-grained, repository-specific tokens.
- CI/CD IAM policies should be narrowed to least-privilege before production rollout.
- OpenTelemetry exporter configuration should be finalized for full distributed tracing.

10. Deliverables (FINAL package)

File	Purpose
<code>cli_agent.py</code>	CLI coding agent with tool-calling and human-in-the-loop confirmation
<code>cli_auth.py</code>	OAuth 2.0 PKCE login/logout for the CLI, keyring-based token storage
<code>repo_tools.py</code>	GitHub + GitLab repo-management tools (branches, commits, PRs/MRs, issues)
<code>lambda_function.py</code>	Web app backend: auth, rate limiting, EC2 auto-start, streaming
<code>index.html</code>	Web app frontend with login, progress bar, and chat UI
<code>template.yaml</code>	AWS SAM infrastructure-as-code template
<code>requirements.txt</code>	Python dependencies
<code>config_guide.md</code>	Full AWS setup guide: Cognito, DynamoDB, SAM, CI/CD OIDC, repo tokens
<code>.github/workflows/deploy.yml</code>	GitHub Actions deployment pipeline
<code>.gitlab-ci.yml</code>	GitLab CI deployment pipeline

This report and the accompanying coding_agent_platform_code_FINAL.zip package represent the final, delivered state of the Qwen3-Coder-14B coding agent platform for this engagement.